Лабораторийн ажил 8

Удамшил

Мэдээлэл, Компьютерын Ухааны тэнхим

Хичээлийн нэр (индекс): Объект хандлагат программчлал (ISCI201)

Лабораторийн хуваарь: Пүрэв гариг 07:40 – 10:05 цаг

Суралцагчдийн нэр: Э.Баттамир А.Баяржавхлан Б.Маргад-Эрдэнэ

Багшийн нэр: М.Золжаргал (Ph.D)

2026.04.30 | МУИС 8-р байр

This хувьсагч

- Программ ажиллах үед тухайн объектын санах ойн хаягийг хадгалдаг далд заагч.
- Гишүүн өгөгдөл ба аргументийн нэр ижил үед классын өгөгдлийг тодорхой зааж өгнө.
- Параметрийн утга гишүүн өгөгдлийг халхлах үед this заагчаар дамжуулан объектод хандана.
- Зөвхөн static бус гишүүн функцүүдэд ашиглагдах тогтмол(constant) хаягтай заагч юм.

```
Circle::Circle(double radius, double crd_1, double crd_2) : Shape2D("Circle") {
    this->radius = radius;
    this->crd_1 = crd_1;
    this->crd_2 = crd_2;
}
```

Polymorphism ойлголт

```
class Shape2D : public Shape {
    public:
        Shape2D() : Shape() {};
        Shape2D(string name) : Shape(name) {};
        virtual double area() const = 0;
        virtual double perimeter() const = 0;
};
```

```
Shape2D* s1 = new Circle(5, 0, 0);
Shape2D* s2 = new Square(4, 0, 0);

cout << s1->area() << endl;
cout << s2->area() << endl;
```

- **Function Overloading** Функц дахин ачаалах
- **Operator Overloading** Оператор дахин ачаалах
- **Method Overriding** Функц дахин тодорхойлох

3. Static гишүүн өгөгдөл

- Классын бүх объектод нийтлэг нэг хувь тоо хэмжээ
- static түлхүүр үгээр зарладаг
- Классын доторх зарлалт — гаднаас тодорхойлолт шаардлагатай
- Объект үүсгэхгүйгээр хандах боломжтой

```
class Shape {
    protected:
        string name;
        static int count; // зарлалт (классын дотор)
};
```

Амьдралын мөчлөг

- Програм эхлэх үед санах ойд байршина → 0 эсвэл өгсөн утгаар
- Программ дуусах хүртэл санах ойд байрших (static storage duration)
- Объект устсан ч static өгөгдөл устдаггүй
- Гаднаас нэг удаа тодорхойлж утга оноодог

```
// shape.cpp — гаднаас нэг удаа тодорхойлно
int Shape::count = 0;
```

Static өгөгдөлд хандах — Классаар

- ClassName::memberName хэлбэрээр хандана
- Объект үүсгэхгүйгээр хандах боломжтой — хамгийн зөв арга
- Классын нэр ашиглах нь кодыг уншихад тодорхой болгодог

```
// Классын нэрээр дамжуулж хандах
cout << Shape::count; // ✓ тодорхой, зөв арга

// Static функцийг классаар дуудах
int n = Shape::getCount();
```

Static өгөгдөлд хандах — Объектоор

- obj.memberName хэлбэрээр хандах боломжтой
- Тухайн объектын хувийн өгөгдөл биш — нийтлэг хэвээр байна
- Объектоор хандах нь буруу ойлголт өгдөг тул зайлсхийх хэрэгтэй

```
Circle c1(5.0, 0, 0);
cout << c1.getCount(); // объектоор — ажиллана, гэхдээ тодорхойгүй
cout << Shape::getCount(); // классаар — ✓ зөв, тодорхой
```

Static гишүүн функц — Зарлалт

- static түлхүүр үгийг зарлалтад (header) бичнэ
- Тодорхойлолтод (cpp файл) static бичихгүй
- this заагч байхгүй — зөвхөн static гишүүнд хандана
- Объект байхгүйгээр дуудагдах боломжтой

```
// oop.h — зарлалт
class Shape {
static int count;
public:
static int getCount(); // static гэж бичнэ
};
// shape.cpp — тодорхойлолт (static гэж бичихгүй)
int Shape::getCount() { return count; }
```

Static гишүүн функц – Дуудалт

Классаар дуудах

```
// Объект шаардахгүй
int n = Shape::getCount();
cout << n;
```

Объектоор дуудах

```
// Ажиллана, гэхдээ
// тодорхойгүй арга
Circle c;
c.getCount();
```

Онцлог хязгаарлалтууд:

- this заагч байхгүй тул энгийн гишүүн өгөгдөлд хандаж чадахгүй
- Зөвхөн static гишүүн өгөгдөл болон static функцийг л дуудах боломжтой

Static гишүүн өгөгдөлд хандах

```
int obj_count1() { return Shape::getCount(); }


int obj_count2() {
    Shape s;
    return s.count;
}
```

Static гишүүн функц зарлах

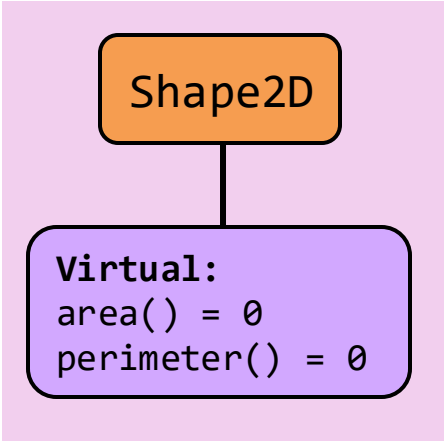
```
static int getCount();

int Shape::getCount() { return count; }
```

Жинхэнэ хийсвэр функц

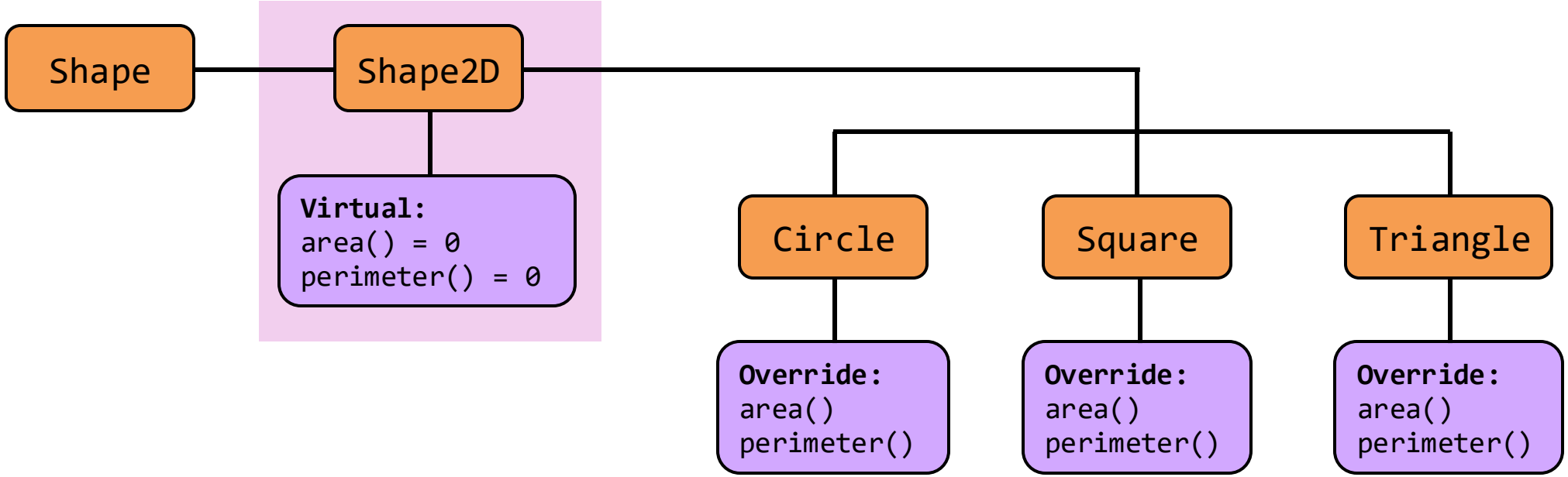
```
virtual double area() const = 0;
virtual double perimeter() const = 0;
```

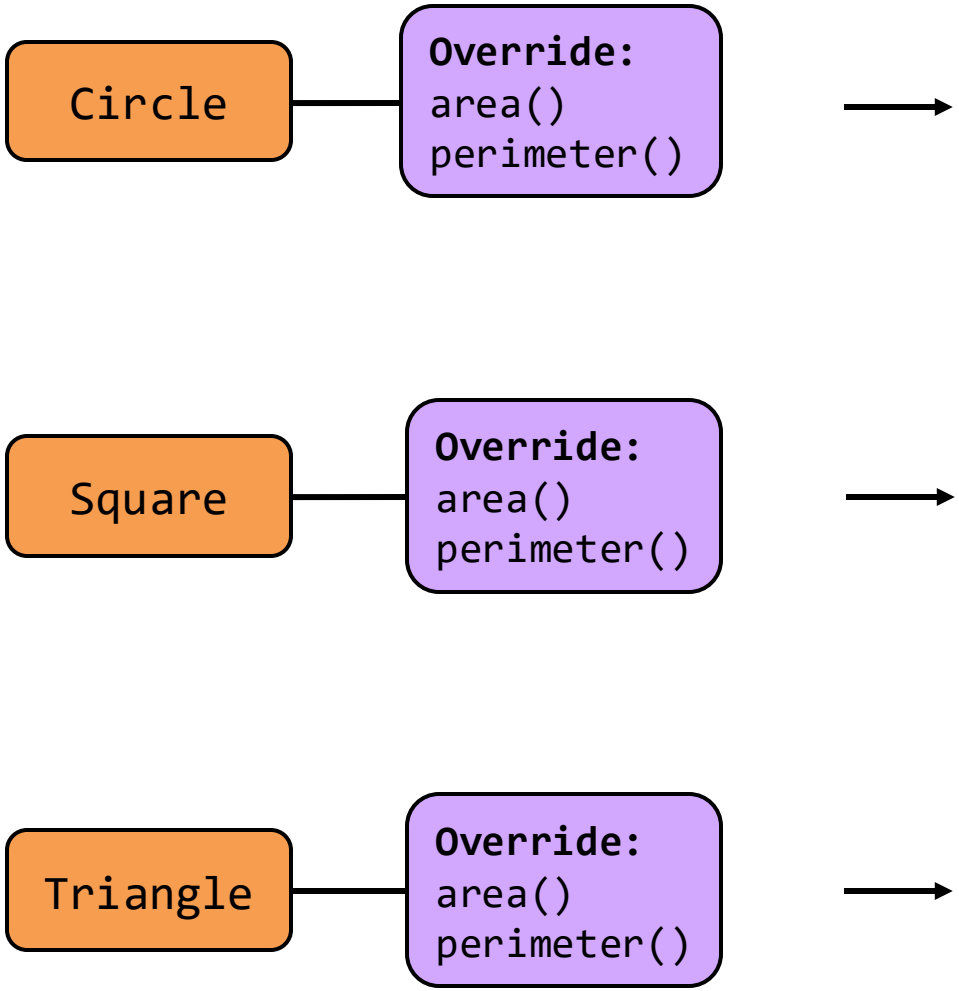
Жинхэнэ хийсвэр функц гэдэг нь C++ хэл дээр зөвхөн зарлагддаг боловч хэрэгжүүлэлт нь байхгүй функц. Ийм функцийг virtual түлхүүр үг ашиглан зарлаж, = 0 гэж тэмдэглэдэг.



Хийсвэр класс

```
class Shape2D : public Shape {
    public:
        Shape2D() : Shape() {};
        Shape2D(string name) : Shape(name) {};
        virtual double area() const = 0;
        virtual double perimeter() const = 0;
};
```






```
double Circle::area() const {
    return M_PI * this->radius * this->radius;
}

double Circle::perimeter() const {
    return 2 * M_PI * this->radius;
}
```

```
double Square::area() const {
    return this->side * this->side;
}

double Square::perimeter() const {
    return 4 * this->side;
}
```

```
double Triangle::area() const {
    return (sqrt(3)/4) * this->side * this->side;
}

double Triangle::perimeter() const {
    return 3 * this->side;
}
```

Хэрэгжүүлэлт: Үүссэн объектийг тоолох статик хувьсагч

```
class Shape {
    protected:
        string name;
        static int count;
```

```
Shape::Shape() : name("Shape") { Shape::count++; }
Shape::Shape(const string& name) : name(name) { Shape::count++; }
int Shape::getCount() { return count; }
```

```
#include "../oop.h"
int obj_count() { return Shape::getCount(); }
```

Хэрэгжүүлэлт: this хувьсагч ашиглах

```
Triangle::Triangle(double side, double x0, double y0) : Shape2D("Triangle")
{
    this->side = side;
    double height = (sqrt(3) / 2) * side;


    this->x0 = x0;
    this->y0 = y0;


    this->x1 = this->x0 - this->side / 2;
    this->y1 = this->y0 - height;


    this->x2 = this->x0 + this->side / 2;
    this->y2 = this->y0 - height;
}
```

Хэрэгжүүлэлт: Shape2D класс төрөлтэй объектуудыг заах заагч хүснэгт

```
int cap = 2;
int siz = 0;


Shape2D **shapes = new Shape2D*[cap];


while (true) {
    cout << "1. Make circle   2. Make square   3. Make triangle   4. Exit" << endl;
    cin >> k;

    double side, radius;

    if (k == 4) break;
```

Хэрэгжүүлэлт: Заагч хүснэгтийн хэмжээг динамикаар удирдах.

```
if (siz == cap) {
    cap *= 2;


    Shape2D **newArr = new Shape2D*[cap];


    for (int i = 0; i < siz; i++) {
        newArr[i] = shapes[i];
    }


    delete[] shapes;
    shapes = newArr;
}
```

Хэрэгжүүлэлт: Shape2D төрөлтэй объектуудыг талбайгаар эрэмбэлэх.

```
void sortArea(Shape2D* list[]) {
    cout << endl;
    for (int i = 1; i < obj_count(); i++) {
        Shape2D* key = list[i];
        int j = i - 1;


        while (j >= 0 && list[j]->area() > key->area()) {
            list[j + 1] = list[j];
            j--;
        }
        list[j + 1] = key;
    }
    cout << "Sorted by Area..." << endl;
}
```

Хэрэгжүүлэлт: main() функцад ашиглах.

```
sortArea(shapes);
print(shapes);
sortPerimeter(shapes);
print(shapes);
cout << endl << obj_count() << " objects created." << endl;
```

Хэрэгжүүлэлт: Программын гаралт.

```
defint@defint-Inspiron-15-3530:~/num/oop/oop/lab8$ ./go
1. Make circle   2. Make square   3. Make triangle   4. Exit
1
Radius: 3
1. Make circle   2. Make square   3. Make triangle   4. Exit
2
Side: 3
1. Make circle   2. Make square   3. Make triangle   4. Exit
3
Side: 3
1. Make circle   2. Make square   3. Make triangle   4. Exit
4

Sorted by Area...
Name: Triangle    Area: 3.89711    Perimeter: 9
Name: Square    Area: 9    Perimeter: 12
Name: Circle    Area: 28.2743    Perimeter: 18.8496

Sorted by Perimeter...
Name: Triangle    Area: 3.89711    Perimeter: 9
Name: Square    Area: 9    Perimeter: 12
Name: Circle    Area: 28.2743    Perimeter: 18.8496

3 objects created.
```

Эх сурвалж

1. М. Золжаргал багшийн лекцийн материал
2. Объект хандлагат технологийн С++ программчал, Ж. Пүрэв, 2008, Улаанбаатар
3. geeksforgeeks